

Fullstack + QA Automation Assessment Report

Candidate: Jyoti Jadhav (JYOOO) | **Company:** Streamhub LTD | **Section:** A – Web Application + UI Automation | **Project:** Loan Dashboard (fullstack-qa-automation-assessment)

1. Summary

This report documents my submission for Section A. I built a Loan Dashboard web application with an EMI calculator, a searchable loan report and a chart. I then automated it with Playwright + Cucumber (TypeScript, Page Object Model), added JSONPlaceholder API tests, solved two SQL scenarios, and documented an AI-assisted self-healing locator approach. Evidence (screenshots, Playwright report, SQL result files) is stored in the repository.

Area	Deliverable	Status
A1	Web application: dashboard, EMI calculator, report, chart	Done
A2	Playwright + Cucumber UI automation with POM	Done
A3	API boundary and negative tests (JSONPlaceholder)	Done - 3/3 passed
A4	SQL Scenario 1 (round-trip transfers) and Scenario 2 (IPL streak)	Done
Extra	AI-SELF-HEALING.md, HTML report, README with AI usage reflection	Done

2. Tech Stack

- **Frontend:** Vite-based web app (HTML/JavaScript), bar chart for loan portfolio
- **UI automation:** Playwright, Cucumber, TypeScript, Page Object Model, .env-based configuration
- **API automation:** Playwright API tests against <https://jsonplaceholder.typicode.com/posts>
- **SQL:** SQLite-style schema and queries, run through sql/run_queries.py, results saved in sql/results/

- **Reporting:** Playwright HTML report (npm run report)

3. A1 – Web Application

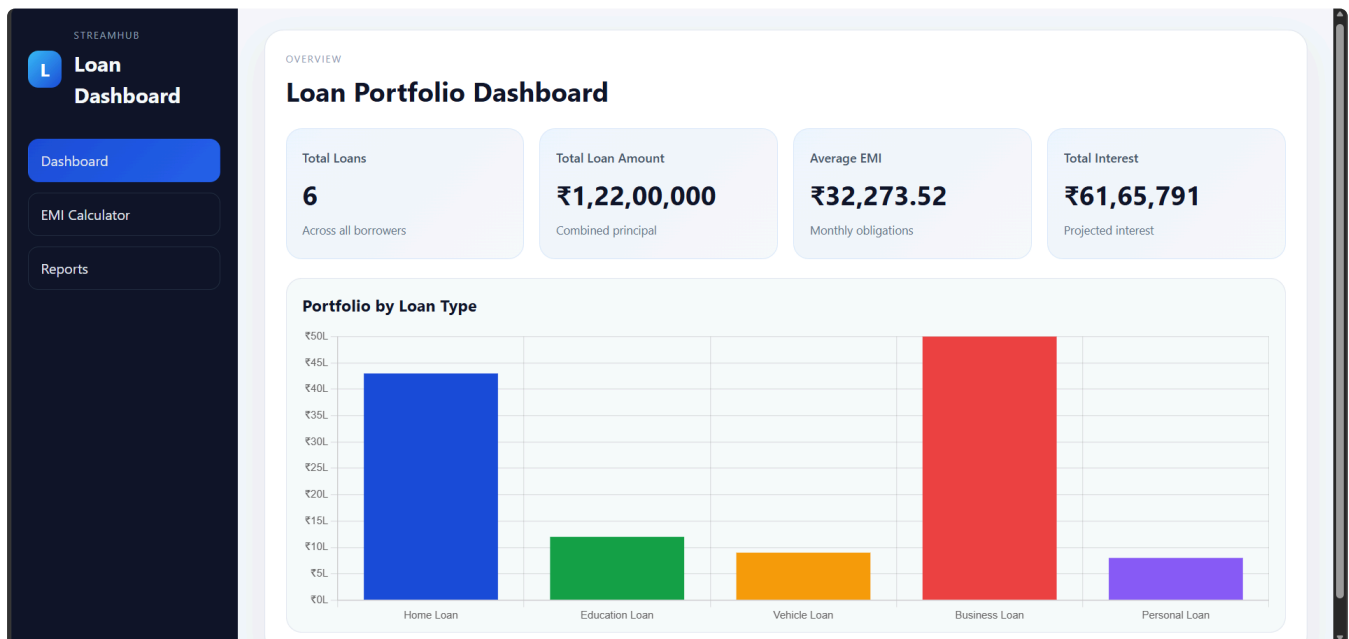
The app has a dark sidebar (Dashboard, EMI Calculator, Reports) under the Streamhub "Loan Dashboard" branding.

3.1 Dashboard

The Loan Portfolio Dashboard shows four summary cards and a "Portfolio by Loan Type" bar chart (Home, Education, Vehicle, Business and Personal loans). The chart is visible and carries non-zero data.

Metric	Value shown
Total Loans	6
Total Loan Amount	₹1,22,00,000
Average EMI	₹32,273.52
Total Interest	₹61,65,791

The totals are consistent with the report table: the six principals add up to ₹1,22,00,000, and the six EMIs (₹193,641 combined) average to about ₹32,273.5.



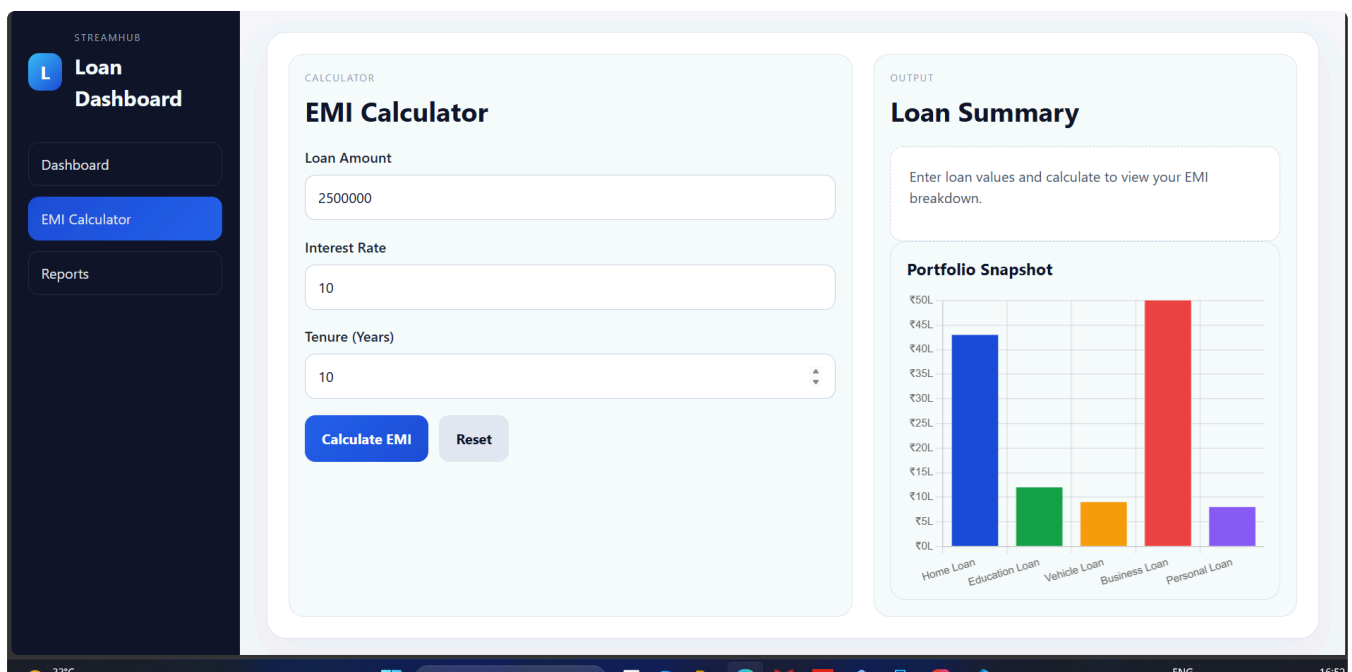
3.2 EMI Calculator

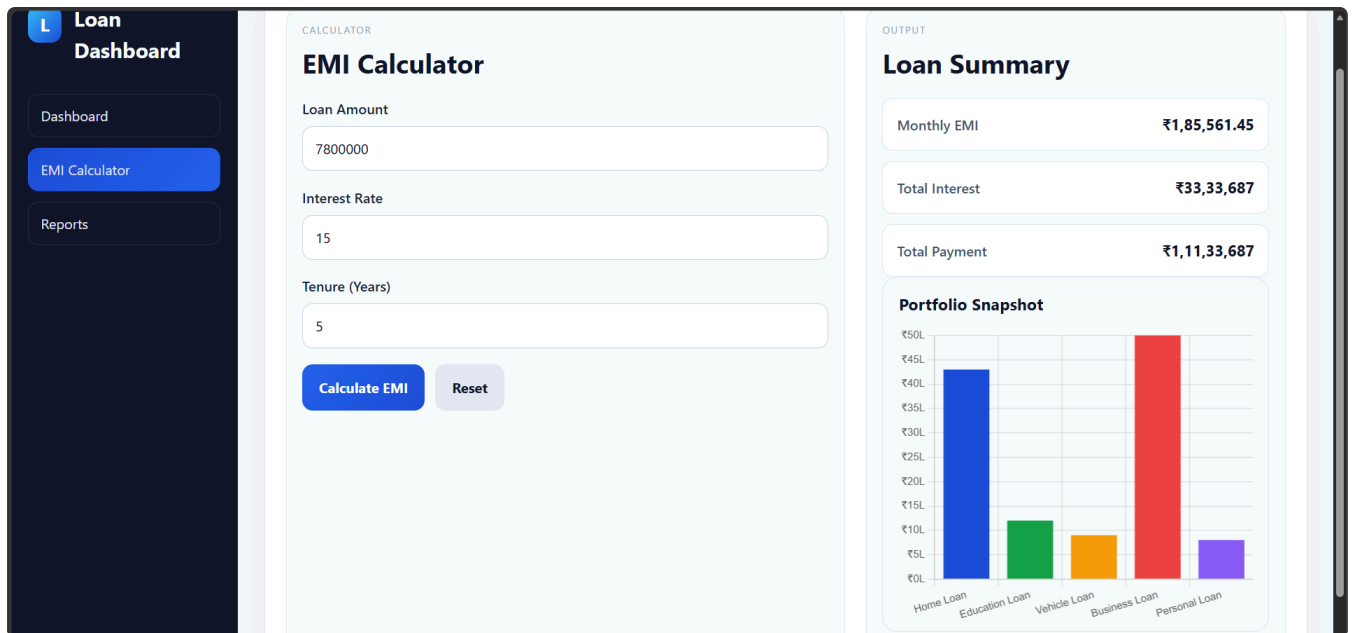
Inputs are Loan Amount, Interest Rate and Tenure (Years), with Calculate EMI and Reset buttons. Before calculation the Loan Summary panel shows a prompt asking the user to enter values, and the Portfolio Snapshot chart stays visible.

Worked example from the screenshots: Loan ₹78,00,000, rate 15%, tenure 5 years.

Output	Displayed value
Monthly EMI	₹1,85,561.45
Total Interest	₹33,33,687
Total Payment	₹1,11,33,687

Cross-check: $₹1,85,561.45 \times 60 \text{ months} = ₹1,11,33,687$, and total payment – principal = ₹33,33,687, so the three outputs agree with each other.





Formula used: $EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)$, where P is the principal, r the monthly rate (annual rate / 12 / 100) and n the number of months. A zero-interest case is handled separately ($EMI = P / n$). The automation computes the expected value on its own and compares it with the value on screen.

3.3 Reports (input-driven view)

The Loan Portfolio Report lists six loans with a text search (customer or loan type) and a Status filter.

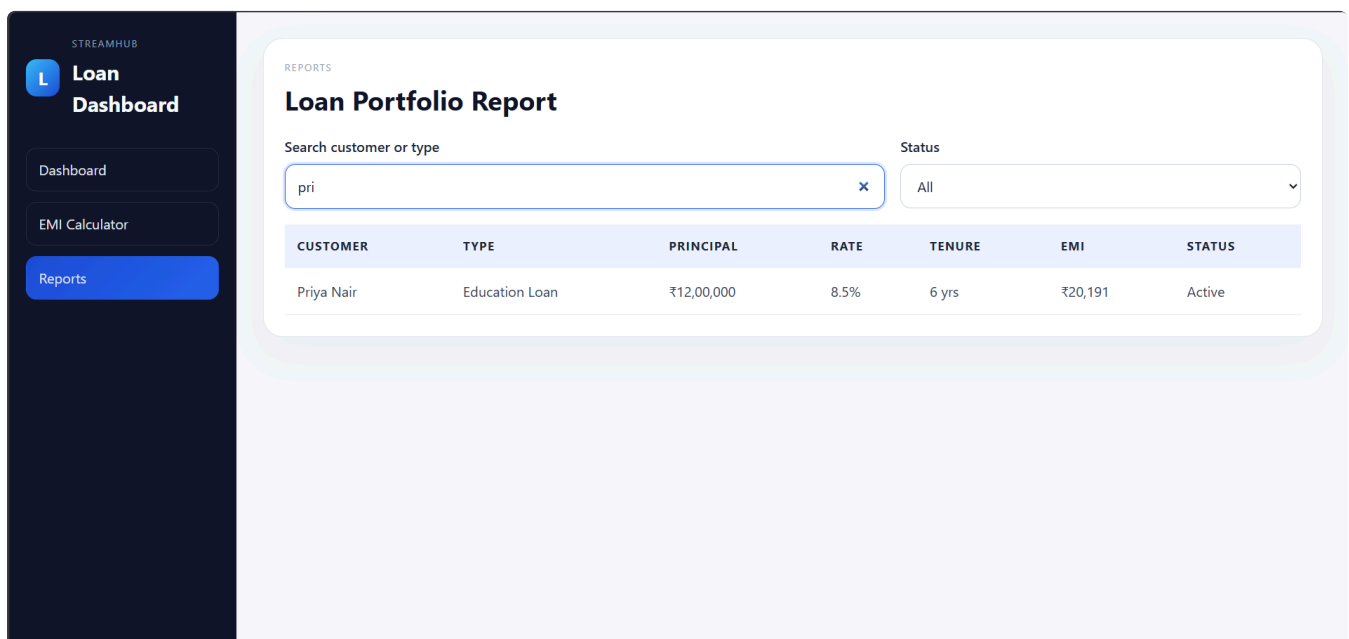
Customer	Type	Principal	Rate	Tenure	EMI	Status
Aarav Sharma	Home Loan	₹25,00,000	10%	10 yrs	₹31,731	Active
Priya Nair	Education Loan	₹12,00,000	8.5%	6 yrs	₹20,191	Active
Rohit Mehta	Vehicle Loan	₹9,00,000	7.25%	5 yrs	₹17,860	Closed
Sneha Verma	Business Loan	₹50,00,000	12%	8 yrs	₹83,328	Active
Vikram Joshi	Personal Loan	₹8,00,000	11.5%	4 yrs	₹20,665	Pending
Meera Kulkarni	Home Loan	₹18,00,000	9.25%	12 yrs	₹19,866	Active



The screenshot shows a web application interface for a 'Loan Portfolio Report'. On the left is a dark sidebar with the 'STREAMHUB' logo and a 'Loan Dashboard' section containing links to 'Dashboard', 'EMI Calculator', and 'Reports' (which is highlighted in blue). The main content area has a light blue header with 'REPORTS' and 'Loan Portfolio Report'. Below the header is a search bar labeled 'Search customer or type' and a 'Status' dropdown menu set to 'All'. The search bar contains the text 'Search by name or loan type'. Below these is a table with 7 columns: CUSTOMER, TYPE, PRINCIPAL, RATE, TENURE, EMI, and STATUS. The table contains 7 rows of loan data.

CUSTOMER	TYPE	PRINCIPAL	RATE	TENURE	EMI	STATUS
Aarav Sharma	Home Loan	₹25,00,000	10%	10 yrs	₹31,731	Active
Priya Nair	Education Loan	₹12,00,000	8.5%	6 yrs	₹20,191	Active
Rohit Mehta	Vehicle Loan	₹9,00,000	7.25%	5 yrs	₹17,860	Closed
Sneha Verma	Business Loan	₹50,00,000	12%	8 yrs	₹83,328	Active
Vikram Joshi	Personal Loan	₹8,00,000	11.5%	4 yrs	₹20,665	Pending
Meera Kulkarni	Home Loan	₹18,00,000	9.25%	12 yrs	₹19,866	Active

Search test: typing "pri" filters the table to a single row, Priya Nair (Education Loan, ₹12,00,000, 8.5%, 6 yrs, ₹20,191, Active). This confirms the filter is dynamic and partial-match.



This screenshot shows the same 'Loan Portfolio Report' dashboard as the previous one, but with the search filter 'pri' applied in the search bar. The search bar now contains 'pri' and a clear button (X). The table now displays only one row, for Priya Nair, as the search results are filtered based on the partial match.

CUSTOMER	TYPE	PRINCIPAL	RATE	TENURE	EMI	STATUS
Priya Nair	Education Loan	₹12,00,000	8.5%	6 yrs	₹20,191	Active

4. A2 – UI Automation (Playwright + Cucumber)

Flow: Feature file → Step definitions → Page Objects → Playwright → Web application.

Structure: automation/features, pages, steps, support and utils; configuration in cucumber.cjs, playwright.config.ts and .env (BASE_URL).

Scenarios covered:

- Dashboard loads and navigation between Dashboard, EMI Calculator and Reports works

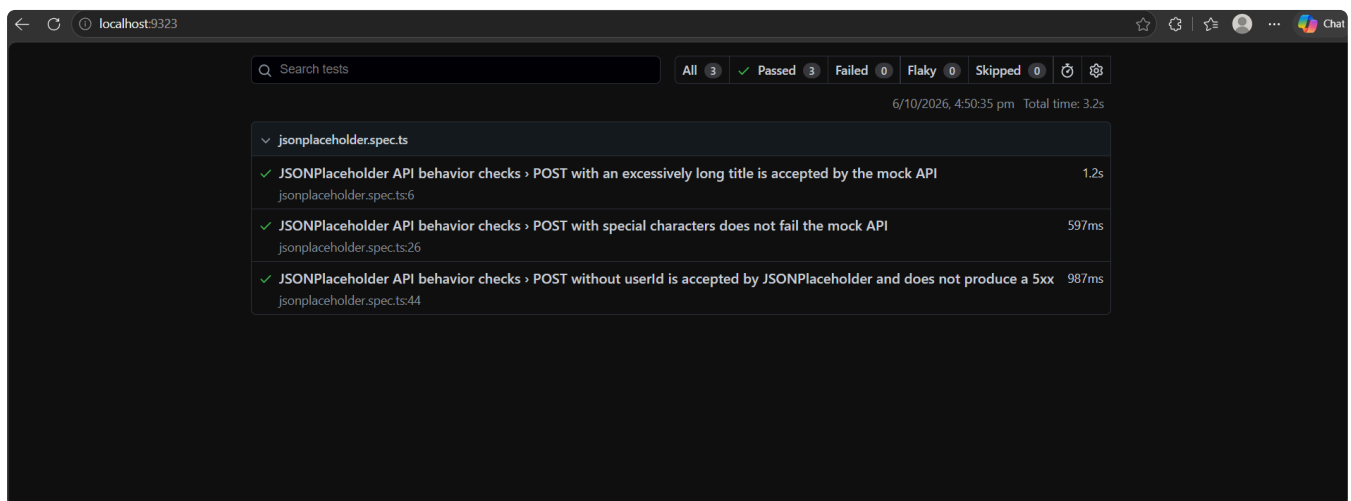
- EMI calculation with an independently computed expected value (including zero interest)
- Report search and filter behaviour (for example searching "pri")
- Chart is visible, renders correctly and contains non-zero data

Locator strategy: Preferred locators are getByRole, getByLabel, getByText and getId. Avoided are nth(), deep CSS selectors, absolute XPath and any position-dependent selector.

Result evidence: no screenshot of the Cucumber UI run was shared with this report. The README's example figures (4 scenarios, 30 steps) must match the latest real run before submission.

5. A3 – API Automation (JSONPlaceholder)

Run with npm run api:test. Playwright HTML report (6/10/2026, 4:50:35 pm): **3 tests, 3 passed, 0 failed, 0 flaky, 0 skipped. Total time 3.2 s.**



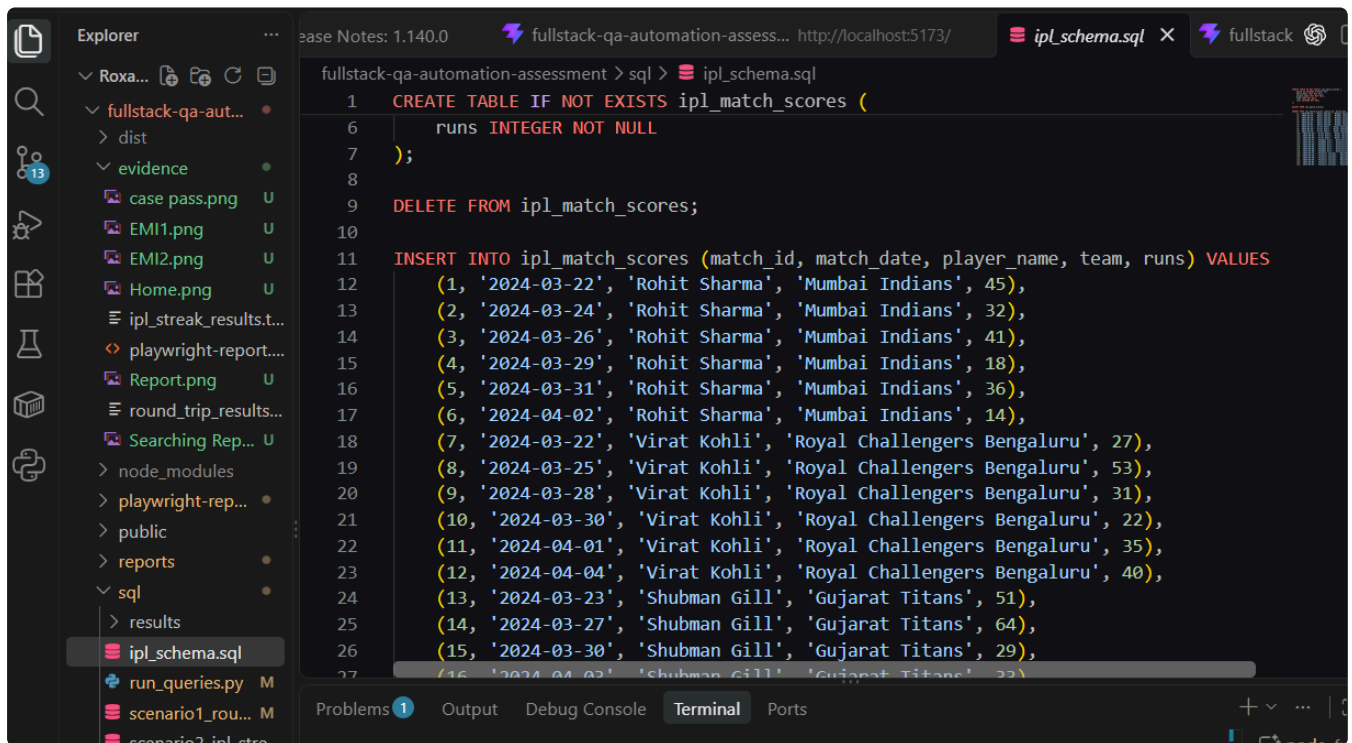
#	Test	Time
1	POST with an excessively long title is accepted by the mock API	1.2 s
2	POST with special characters does not fail the mock API	597 ms
3	POST without userId is accepted by JSONPlaceholder and does not produce a 5xx	987 ms

Key observation: JSONPlaceholder is a mock API and accepts payloads a production API would reject. The tests therefore record the actual observed behaviour instead of assuming a 400 response. The main safety assertion is that invalid input never causes a

5xx server error.

6. A4 - SQL Scenarios

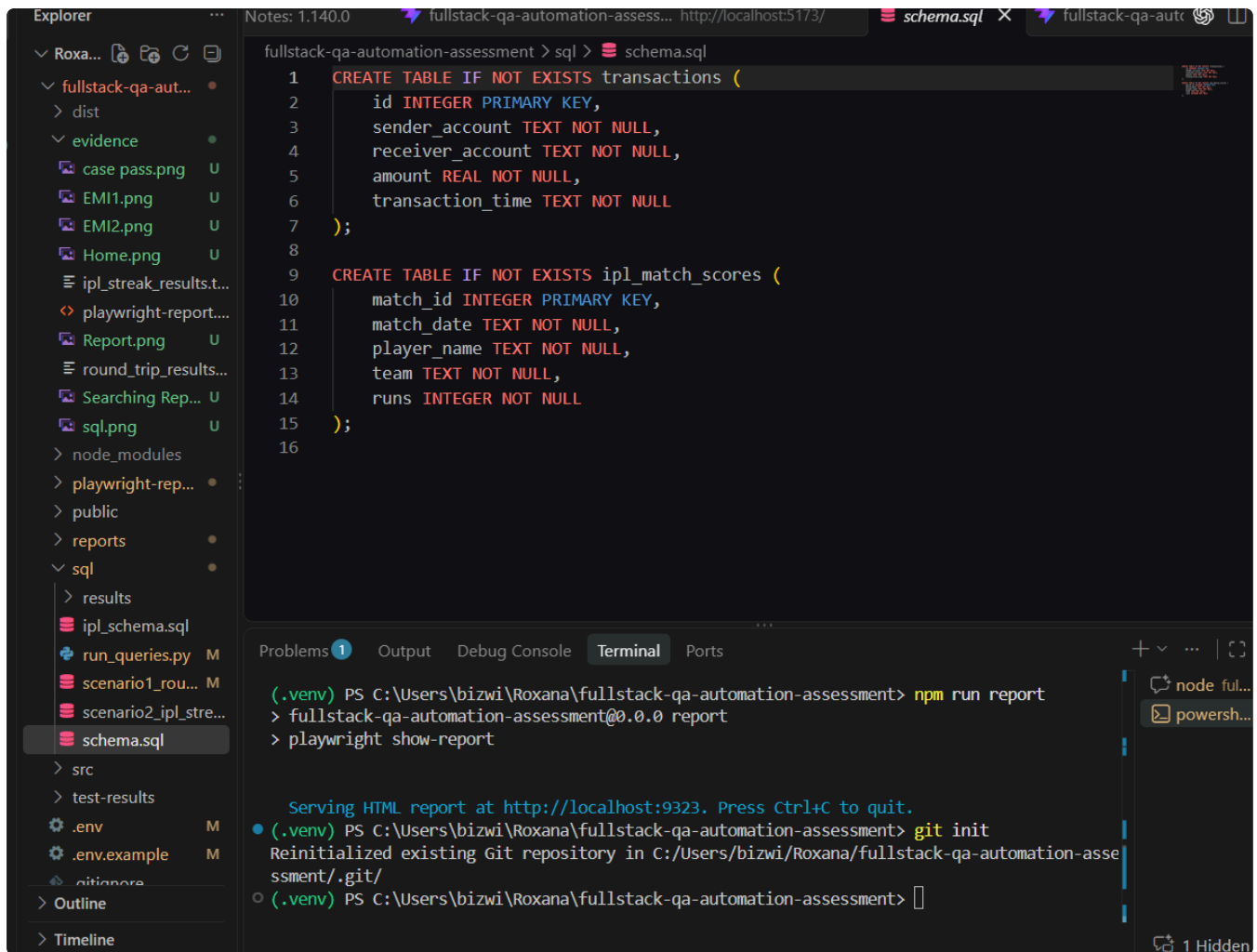
Schema files: sql/schema.sql (tables transactions and ipl_match_scores), sql/ipl_schema.sql (IPL sample data). Queries: scenario1_round_trip.sql and scenario2_ipl_streak.sql, executed by python sql/run_queries.py with output saved to sql/results/ (round_trip_results and ipl_streak_results).



```

1 CREATE TABLE IF NOT EXISTS ipl_match_scores (
6     runs INTEGER NOT NULL
7 );
8
9 DELETE FROM ipl_match_scores;
10
11 INSERT INTO ipl_match_scores (match_id, match_date, player_name, team, runs) VALUES
12 (1, '2024-03-22', 'Rohit Sharma', 'Mumbai Indians', 45),
13 (2, '2024-03-24', 'Rohit Sharma', 'Mumbai Indians', 32),
14 (3, '2024-03-26', 'Rohit Sharma', 'Mumbai Indians', 41),
15 (4, '2024-03-29', 'Rohit Sharma', 'Mumbai Indians', 18),
16 (5, '2024-03-31', 'Rohit Sharma', 'Mumbai Indians', 36),
17 (6, '2024-04-02', 'Rohit Sharma', 'Mumbai Indians', 14),
18 (7, '2024-03-22', 'Virat Kohli', 'Royal Challengers Bengaluru', 27),
19 (8, '2024-03-25', 'Virat Kohli', 'Royal Challengers Bengaluru', 53),
20 (9, '2024-03-28', 'Virat Kohli', 'Royal Challengers Bengaluru', 31),
21 (10, '2024-03-30', 'Virat Kohli', 'Royal Challengers Bengaluru', 22),
22 (11, '2024-04-01', 'Virat Kohli', 'Royal Challengers Bengaluru', 35),
23 (12, '2024-04-04', 'Virat Kohli', 'Royal Challengers Bengaluru', 40),
24 (13, '2024-03-23', 'Shubman Gill', 'Gujarat Titans', 51),
25 (14, '2024-03-27', 'Shubman Gill', 'Gujarat Titans', 64),
26 (15, '2024-03-30', 'Shubman Gill', 'Gujarat Titans', 29),
27 (16, '2024-04-03', 'Shubman Gill', 'Gujarat Titans', 23)

```



Scenario 1 – Round-trip transfers

Finds transactions where account A sends money to B and B sends money back to A, with both transfers within 24 hours and the two amounts within 10% of each other. Implemented with a **self-join** on the transactions table (sender_account, receiver_account, amount, transaction_time).

Scenario 2 – IPL scoring streak

Finds players with 30+ runs in at least 3 consecutive matches and returns the player name and the streak start date. Implemented with **window functions** (row numbering per player compared with row numbering over the 30+ matches to group consecutive runs).

Sample data check: Rohit Sharma scored 45, 32 and 41 in his first three matches, so his streak starts on 22 March 2024. Virat Kohli's 30+ scores (53, 31, then 35, 40) are broken by a 22, so he has no 3-match streak in the visible rows.

7. AI-Assisted Self-Healing Locators

AI-SELF-HEALING.md demonstrates the repair workflow on intentionally brittle locators:

Broken locator → failure detection → DOM/accessibility context → AI analysis → candidate locator → validation → safe update.

Brittle selectors (position-based or deep CSS) are replaced with semantic ones such as `getByRole('button', { name: 'Calculate EMI' })` and `getByLabel('Loan Amount')`. The approach is a proof of concept, not a fully autonomous repair pipeline.

8. AI Usage Reflection

Used AI for: project structure, Playwright/Cucumber framework, page objects, locator strategy, API boundary-case ideas, SQL window-function logic, debugging and documentation.

Validated manually: application-specific selectors, expected EMI values, real API behaviour, test run results, environment configuration and final locator reliability.

AI was treated as an engineering assistant. Every suggestion was run and checked against the application before it went into the final code.

9. How to Run

1. npm install
2. npx playwright install chromium
3. Copy .env.example to .env (BASE_URL=http://localhost:5173)
4. Terminal 1: npm run dev -- --host 0.0.0.0
5. Terminal 2: npm run ui:test, npm run api:test, or npm run test for both
6. npm run report to open the Playwright HTML report
7. python sql/run_queries.py for the SQL results

10. Evidence Index

File	Shows
evidence/Home.png	Dashboard with summary cards and chart
evidence/EMI1.png	EMI Calculator before calculation
evidence/EMI2.png	EMI result: ₹1,85,561.45 for ₹78,00,000 at 15% for 5 yrs
evidence/Report.png	Full Loan Portfolio Report
evidence/Searching Report.png	Search "pri" returning Priya Nair

File	Shows
evidence/case pass.png	Playwright HTML report: 3 of 3 API tests passing
evidence/sql.png	ipl_schema.sql sample data in VS Code
playwright-report	HTML report: 3/3 API tests passed
sql/results	Round-trip and IPL streak result files

11. Limitations and Future Work

Limitations: JSONPlaceholder does not validate input like a real API; SQL scenarios run on controlled sample data; self-healing is a proof of concept.

Future work: GitHub Actions CI, cross-browser and parallel runs, Dockerised test environment, Allure reporting, an automated AI locator-repair pipeline, visual regression and accessibility testing.

12. Final Checks Before Submitting

- Update the README test-result numbers to match the latest real run (the current ones are marked as an example).
- The GitHub page shows the repository as 97.7% HTML. Confirm the TypeScript, SQL and Python sources are committed, and that evidence/ and sql/results are included.
- Confirm .env is not committed and .env.example is.
- The README "Reopen and Run Again" section contains a local Windows path (C:\Users...). Remove it before sharing publicly.
- Add a unit label to the Interest Rate field (for example "Interest Rate (%)") so it matches the "Tenure (Years)" label.
- The terminal in the screenshot shows "git init" re-initialising the repo and 13 pending changes; commit and push so the GitHub repo is current.